

Sistema de Gestión Académica — Plan de Proyecto

Información General

Campo	Detalle
Proyecto	Sistema de Gestión Académica
Tipo	Demostrativo — Guiado por el Docente
Stack	NestJS + Next.js + PostgreSQL + Docker + Prisma
Repositorio	jaquimbayoc7/gestion-academica-sistema

Casos de Uso del Proyecto

#	Caso de Uso
CU-01	Registrar estudiante con datos personales, código estudiantil, correo institucional y programa académico.
CU-02	Registrar docente con datos personales, título profesional, especialidad y correo institucional.
CU-03	Crear asignaturas con nombre, código, número de créditos y programa al que pertenece.
CU-04	Matricular estudiante en una asignatura de un período verificando que no exista duplicidad.
CU-05	Registrar calificaciones (nota 1, nota 2, nota 3, nota definitiva) de un estudiante en una matrícula.

Plan de Releases

Release 1 — Segundo Corte: Backend y Frontend

Objetivo: Entregar la API REST completa con arquitectura en capas (Controller → Service → Repository) y el frontend con las vistas de CRUD para todas las entidades base.

Sprint	Período	Festivos	Alcance	Historias de Usuario
Sprint 1 — Infraestructura y entidades base	Mar 16 → Mar 29	Mar 23 (San José)	Docker Compose, Prisma schema, migraciones, módulos CRUD de Estudiante, Docente y ProgramaAcademico	HU-01, HU-02, HU-03
Sprint 2 — Entidades académicas y cross-cutting	Mar 30 → Abr 10	Abr 2 (Jueves Santo), Abr 3 (Viernes Santo)	Módulos CRUD de Asignatura, PeriodoAcademico, AsignacionDocente. Common module (Filters, Pipes, Interceptors)	HU-04, HU-05, HU-06
Sprint 3 — Matrícula, Calificaciones y Frontend base	Abr 13 → Abr 17	—	Módulos de Matrícula y Calificación con lógica de negocio. Frontend: estructura Next.js, listados y formularios de entidades base	HU-07, HU-08, HU-09

■ Cierre Segundo Corte: 17 de Abril de 2026

Release 2 — Tercer Corte: Integración y Despliegue

Objetivo: Integración completa frontend ↔ backend, formularios avanzados con relaciones, registro de calificaciones desde la interfaz y despliegue funcional con Docker.

Sprint	Período	Festivos	Alcance	Historias de Usuario
--------	---------	----------	---------	----------------------

Sprint 4 — Frontend avanzado e integración	Abr 20 → May 8	May 1 (Día del Trabajo)	Formularios con relaciones (selects dinámicos), páginas de detalle, navegación completa, estados de carga y error	HU-10
Sprint 5 — Cierre y despliegue	May 11 → May 22	May 18 (Día de la Ascensión)	Integración de flujos completos (matricular → calificar), pruebas de integración, despliegue con Docker Compose	HU-11

■ Cierre Tercer Corte: 22 de Mayo de 2026

Historias de Usuario

HU-01 — Gestión de Estudiantes (CU-01)

Como administrador académico, **quiero** registrar, consultar, editar y eliminar estudiantes, **para** mantener actualizada la información de los estudiantes de la institución.

Criterios de Aceptación

- [] Se puede crear un estudiante con: nombres, apellidos, código estudiantil, documento de identidad, correo institucional, fecha de nacimiento y programa académico.
- [] El código estudiantil y el documento de identidad son únicos; si se duplican, el sistema retorna un error claro.
- [] El correo institucional debe tener formato válido.
- [] Se puede consultar la lista de todos los estudiantes con paginación.
- [] Se puede consultar un estudiante por su ID y ver sus datos completos incluyendo el programa académico asociado.
- [] Se puede editar la información de un estudiante existente.
- [] Se puede eliminar (soft delete o hard delete) un estudiante que no tenga matrículas asociadas.
- [] Si se intenta eliminar un estudiante con matrículas, el sistema retorna un error descriptivo.

Tareas Técnicas

- Backend: `CreateEstudianteDto`, `UpdateEstudianteDto` con class-validator
- Backend: `EstudianteRepository` (Prisma queries)
- Backend: `EstudianteService` (lógica de negocio + validaciones)

- Backend: `EstudianteController` (endpoints GET, GET/:id, POST, PUT/:id, DELETE/:id)
- Frontend: Página de listado `/estudiantes`
- Frontend: Formulario de creación `/estudiantes/new`
- Frontend: Página de detalle `/estudiantes/[id]`

HU-02 — Gestión de Docentes (CU-02)

Como administrador académico, **quiero** registrar, consultar, editar y eliminar docentes, **para** tener un registro centralizado del cuerpo docente de la institución.

Criterios de Aceptación

- [] Se puede crear un docente con: nombres, apellidos, documento de identidad, título profesional, especialidad, correo institucional y teléfono.
- [] El documento de identidad y el correo institucional son únicos.
- [] Se puede consultar la lista de docentes.
- [] Se puede consultar un docente por ID con sus datos completos.
- [] Se puede editar la información de un docente existente.
- [] Se puede eliminar un docente que no tenga asignaciones activas; caso contrario, se retorna error.

Tareas Técnicas

- Backend: DTOs, Repository, Service, Controller del módulo Docente
- Frontend: Listado, formulario y detalle de docentes

HU-03 — Gestión de Programas Académicos (Soporte CU-01, CU-03)

Como administrador académico, **quiero** gestionar los programas académicos de la institución, **para** asociar estudiantes y asignaturas a su programa correspondiente.

Criterios de Aceptación

- [] Se puede crear un programa académico con: nombre, código, facultad y duración en semestres.
- [] El código del programa es único.
- [] Se puede listar todos los programas académicos.
- [] Se puede editar un programa existente.
- [] Se puede eliminar un programa sin estudiantes ni asignaturas asociadas.

Tareas Técnicas

- Backend: DTOs, Repository, Service, Controller del módulo ProgramaAcademico
- Frontend: Listado y formulario de programas

HU-04 — Gestión de Asignaturas (CU-03)

Como coordinador de programa, **quiero** crear y gestionar asignaturas con su información académica, **para** organizar la oferta curricular de cada programa.

Criterios de Aceptación

- [] Se puede crear una asignatura con: nombre, código, número de créditos y programa académico al que pertenece.
- [] El código de asignatura es único.
- [] El programa académico referenciado debe existir; caso contrario, se retorna error.
- [] Se puede listar asignaturas con filtro por programa académico.
- [] Se puede editar y eliminar asignaturas sin matrículas asociadas.

Tareas Técnicas

- Backend: DTOs con validación de FK (programaAcademicId), Repository, Service, Controller
- Frontend: Listado con filtro por programa, formulario con select de programa

HU-05 — Gestión de Períodos Académicos (Soporte CU-04)

Como administrador académico, **quiero** configurar los períodos académicos del año, **para** organizar temporalmente las matrículas, asignaciones y calificaciones.

Criterios de Aceptación

- [] Se puede crear un período con: nombre (ej: "2026-A"), fecha de inicio, fecha de fin y estado (activo/inactivo).
- [] El nombre del período es único.
- [] La fecha de fin debe ser posterior a la fecha de inicio.
- [] Solo puede existir un período activo a la vez.
- [] Se puede listar todos los períodos ordenados por fecha.
- [] Se puede editar y cambiar el estado de un período.

Tareas Técnicas

- Backend: Validación de fechas en Service, lógica de período activo único
- Frontend: Listado con indicador de estado, formulario con date pickers

HU-06 — Asignación de Docente a Asignatura (Soporte CU-04)

Como coordinador de programa, **quiero** asignar un docente a una asignatura en un período académico, **para** definir quién dictará cada materia en cada período.

Criterios de Aceptación

- [] Se puede crear una asignación seleccionando: docente, asignatura y período académico.
- [] No se permite duplicar la misma asignación (mismo docente + asignatura + período).
- [] El docente, la asignatura y el período deben existir previamente.
- [] Se puede listar las asignaciones filtradas por período.
- [] Se puede eliminar una asignación que no tenga matrículas dependientes.

Tareas Técnicas

- Backend: DTO con validación de 3 FKs, unicidad compuesta en Repository
- Frontend: Formulario con 3 selects dinámicos (docente, asignatura, período)

HU-07 — Matrícula de Estudiante en Asignatura (CU-04)

Como estudiante o administrador, **quiero** matricular un estudiante en una asignatura de un período, **para** formalizar la inscripción académica y habilitar el registro de notas.

Criterios de Aceptación

- [] Se puede crear una matrícula seleccionando: estudiante y asignación docente (que incluye asignatura + período).
- [] No se permite matricular al mismo estudiante dos veces en la misma asignatura del mismo período.
- [] El estudiante y la asignación docente deben existir.
- [] La matrícula registra la fecha de inscripción automáticamente.
- [] Se puede consultar las matrículas de un estudiante.
- [] Se puede cancelar una matrícula que no tenga calificaciones registradas.

Tareas Técnicas

- Backend: Validación de unicidad compuesta (estudianteld + asignacionDocenteld), verificación de existencia de FKs
- Frontend: Formulario con selects encadenados (período → asignatura → estudiante)

HU-08 — Registro de Calificaciones (CU-05)

Como docente, **quiero** registrar las calificaciones de los estudiantes matriculados en mis asignaturas, **para** llevar el control del rendimiento académico.

Criterios de Aceptación

- [] Se puede registrar nota 1, nota 2 y nota 3 para una matrícula específica.
- [] Cada nota debe estar en el rango de 0.0 a 5.0.
- [] La nota definitiva se calcula automáticamente como promedio ponderado (configurable: ej. 30%, 30%, 40%).
- [] Solo se puede registrar calificación en matrículas existentes.
- [] Se puede editar las notas mientras el período esté activo.
- [] Se puede consultar las calificaciones de una matrícula.

Tareas Técnicas

- Backend: Validación de rango en DTO, cálculo automático de definitiva en Service
- Frontend: Tabla editable de notas por asignatura con cálculo en tiempo real

HU-09 — Common Module: Filtros, Interceptores y Pipes (Cross-cutting)

Como desarrollador, **quiero** implementar un módulo compartido con filtros de excepción, interceptores de respuesta y pipes de validación, **para** garantizar respuestas consistentes y manejo centralizado de errores en toda la API.

Criterios de Aceptación

- [] Todas las excepciones HTTP retornan un JSON con formato uniforme: `{ statusCode, message, error, timestamp }`.
- [] Todas las respuestas exitosas retornan formato uniforme: `{ statusCode, message, data }`.
- [] El ValidationPipe global está configurado con `whitelist: true` y `forbidNonWhitelisted: true`.
- [] Los errores de validación retornan mensajes claros indicando qué campo falló y por qué.

Tareas Técnicas

- `common/filters/http-exception.filter.ts`
- `common/interceptors/response.interceptor.ts`
- `common/pipes/validation.pipe.ts`
- Registro global en `main.ts`

HU-10 — Frontend: Listados, Formularios y Navegación (CU-01 a CU-05)

Como usuario del sistema, **quiero** gestionar estudiantes, docentes, asignaturas, matrículas y calificaciones desde la interfaz web, **para** administrar el proceso académico sin depender de herramientas externas como Postman.

Criterios de Aceptación

- [] Existe un layout general con sidebar o navbar con enlaces a cada sección (Estudiantes, Docentes, Programas, Asignaturas, Períodos, Matrículas, Calificaciones).
- [] La navegación indica visualmente la sección activa.
- [] El diseño es responsivo (funcional en desktop y tablet).
- [] Existen páginas de listado para cada entidad con datos provenientes de la API.
- [] Existen formularios de creación/edición con validación del lado del cliente.
- [] Los formularios con relaciones usan selects dinámicos (ej: matrícula con selects encadenados: período → asignatura → estudiante).
- [] Los formularios muestran mensajes de error claros si alguna validación falla (duplicidad, campo requerido, rango).
- [] Los formularios muestran estado de carga (spinner/skeleton) mientras se procesan las peticiones.
- [] Tras una operación exitosa, se muestra un mensaje de confirmación y se actualizan los listados.

Tareas Técnicas

- Frontend: `layout.tsx` con componente de navegación (Sidebar o Navbar)
- Frontend: Cliente HTTP centralizado (`lib/api.ts`) y `services/` por entidad
- Frontend: Páginas de listado para cada entidad
- Frontend: Formularios de creación/edición con validación
- Frontend: Página `/calificaciones/[asignacionId]` con tabla editable
- Frontend: Componentes de feedback (toast/alert de éxito/error)
- Frontend: Estilos responsivos con CSS/Tailwind

HU-11 — Integración Final y Despliegue con Docker

Como equipo de desarrollo, **quiero** verificar que todos los servicios funcionen correctamente de forma integrada, **para** garantizar que el sistema se despliega con un solo comando y los flujos completos funcionan de extremo a extremo.

Criterios de Aceptación

- [] Los 3 servicios (PostgreSQL, NestJS, Next.js) se levantan correctamente con `docker compose up`.
- [] El frontend se comunica correctamente con el backend y muestra datos reales de la base de datos.
- [] El flujo completo funciona: crear estudiante → crear asignatura → matricular → registrar calificaciones → consultar notas.
- [] No hay errores de consola ni advertencias críticas en backend ni frontend.
- [] Las migraciones de Prisma se aplican correctamente al iniciar el contenedor.
- [] El README.md del repositorio incluye instrucciones claras de instalación y ejecución.

Tareas Técnicas

- Verificar `docker-compose.yml` con healthchecks y dependencias correctas
- Pruebas de integración del flujo completo
- Documentación del README.md (descripción, tecnologías, instalación, ejecución)
- Revisión de código y limpieza final

Definition of Done (DoD) Global

Cada Historia de Usuario se considera **terminada** cuando cumple **todos** los siguientes criterios:

Backend

- ☐ Endpoint(s) implementados con arquitectura en capas: Controller → Service → Repository.
- ☐ DTOs con validaciones usando `class-validator` y `class-transformer`.
- ☐ Manejo de errores con excepciones HTTP apropiadas (`NotFoundException`, `ConflictException`, `BadRequestException`).
- ☐ Respuestas con formato uniforme (interceptor aplicado).
- ☐ Endpoint probado manualmente con Postman/Thunder Client y funcionando correctamente.

Frontend

- ☐ Página(s) implementada(s) con componentes reutilizables.
- ☐ Consumo del API a través de la capa de `services/`.
- ☐ Manejo de estados: carga (loading), éxito y error.
- ☐ Formularios con validación del lado del cliente.
- ☐ Diseño responsivo y navegable.

Infraestructura y Código

- ☐ Código versionado en GitHub con commits descriptivos.
- ☐ El servicio funciona correctamente con `docker compose up`.
- ☐ No hay errores de consola ni advertencias críticas.
- ☐ Las migraciones de Prisma están aplicadas y el esquema es consistente.

Entidades del Modelo de Datos (Prisma)

Entidad Origen	Cardinalidad	Entidad Destino
Estudiante	1 → N	Matricula
Docente	1 → N	AsignacionDocente
ProgramaAcademico	1 → N	Estudiante
ProgramaAcademico	1 → N	Asignatura
Asignatura	1 → N	AsignacionDocente
PeriodoAcademico	1 → N	AsignacionDocente
AsignacionDocente	1 → N	Matricula
Matricula	1 → 1	Calificacion

Resumen de Entidades

Entidad	Campos principales
Estudiante	id, nombres, apellidos, codigoEstudiantil (unique), documentoidentidad (unique), correoInstitucional (unique), fechaNacimiento, programaAcademicold
Docente	id, nombres, apellidos, documentoidentidad (unique), tituloProfesional, especialidad, correoInstitucional (unique), telefono
ProgramaAcademico	id, nombre, codigo (unique), facultad, duracionSemestres
Asignatura	id, nombre, codigo (unique), creditos, programaAcademicold
PeriodoAcademico	id, nombre (unique), fechaInicio, fechaFin, activo
AsignacionDocente	id, docenteld, asignaturald, periodoAcademicold (unique compound)
Matricula	id, estudianteld, asignacionDocenteld, fechaInscripcion (unique compound: estudianteld + asignacionDocenteld)
Calificacion	id, matriculald (unique), nota1, nota2, nota3, notaDefinitiva

Cronograma de Releases

Release 1 — Segundo Corte (Cierre: 17 Abr 2026) — Backend + Frontend Base

Sprint	Período	Alcance	Festivos
Sprint 1	Mar 16 → Mar 29	Docker, Prisma, Estudiante, Docente, Programa	Mar 23 (San José)
Sprint 2	Mar 30 → Abr 10	Asignatura, Período, Asignación Doc, Filters/Pipes	Abr 2-3 (Semana Santa)
Sprint 3	Abr 13 → Abr 17	Matrícula, Calificación, Common Module, Frontend: listados y forms	—

Release 2 — Tercer Corte (Cierre: 22 May 2026) — Integración + Despliegue

Sprint	Período	Alcance	Festivos
Sprint 4	Abr 20 → May 8	Frontend matrículas, Frontend calificaciones, Navegación y layout, Selects dinámicos	May 1 (Día del Trabajo)
Sprint 5	May 11 → May 22	Integración de flujos, Pruebas de integración, Docker compose validación final, README y documentación	May 18 (Día de la Ascensión)

Festivos Colombianos 2026 (Marzo — Mayo)

Fecha	Festivo	Sprint afectado
Lunes 23 de Marzo	Día de San José	Sprint 1
Jueves 2 de Abril	Jueves Santo	Sprint 2

Viernes 3 de Abril	Viernes Santo	Sprint 2
Viernes 1 de Mayo	Día del Trabajo	Sprint 4
Lunes 18 de Mayo	Día de la Ascensión	Sprint 5

Estado del Proyecto

- [x] **Plan de releases** revisado y aprobado
- [x] **Historias de Usuario** revisadas y aprobadas
- [x] **Criterios de Aceptación** revisados y aprobados
- [x] **Definition of Done** revisado y aprobado
- [x] **Modelo de datos** revisado y aprobado
- [x] **Repositorio GitHub** creado con Issues y Milestones

■ **Repositorio:** github.com/jaquimbayoc7/gestion-academica-sistema ■ **Issues:** 11 HUs + 1 DoD (pinned) ■ **Milestones:** 5 sprints con fechas de cierre